



Module 5

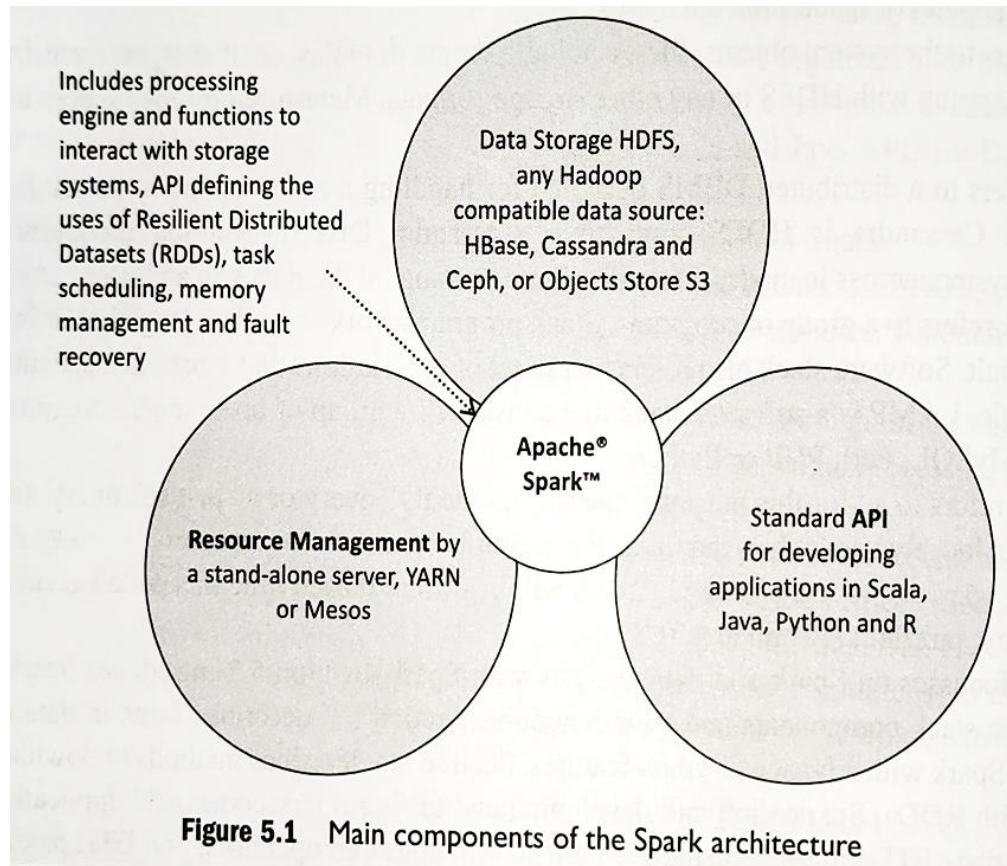
Spark and Big Data Analytics: Spark, Introduction to Data Analysis with Spark.

Text, Web Content and Link Analytics: Introduction, Text Mining, Web Mining, Web Content and Web Usage Analytics, Page Rank, Structure of Web and Analyzing a Web Graph.

TB2: Ch5: 5.2,5.3, Ch 9: 9.1-9.4

- **Apache Spark** is a **fast and general-purpose computing engine** used for big data analytics.
- It can process data **up to 100 times faster** than traditional systems(Mapreduce, RDBMS) and works with **HDFS** (Hadoop Distributed File System) and other compatible data sources.
- Spark uses a **simple and expressive programming model** (RDD, dataframe, dataset) that allows programmers to write less code for complex tasks.
- It is useful for **analytics, ETL (Extract, Transform, Load), machine learning, stream processing, and graph computations**.
- Spark runs on **Windows, Linux, and Mac** systems. It needs **Java** and supports **multiple languages** like **Python, Scala, and R**.
- Spark version 2.3.1 requires:
 - Java 8+**
 - Python 2.7+/3.4+**
 - R 3.1+**
 - Scala 2.11.x**

Spark Architecture



1) Data Storage

- Spark stores data in **HDFS** or any **Hadoop-compatible data sources** like: **HBase, Cassandra, Ceph, or Amazon S3 (Simple Storage Service)**.

2) Standard API

- APIs are available for building applications using: **Scala, Java, Python, R**
- These APIs make it easy for developers to write programs.

3) Resource Management

- Spark can be run on:
 - A **standalone server**, or
 - Cluster managers like **YARN** or **Mesos**.
- These manage the allocation of resources like CPU, memory, etc.



Apache
MESOSTM

Ceph – **Storage system**

- A **reliable, open source storage system** to store **huge amounts of data** (in petabytes or more).
- It **supports HDFS-Compatible storage** and allows **efficient access and distribution**.

Apache Mesos – **Cluster manager**

- A **cluster manager** that **divides and manages system resources** (like CPU and RAM) for running **big data tasks**, including **spark and hadoop jobs**.

S3 (Simple Storage Service) – **Cloud Storage Service**

- A **cloud storage service from Amazon** where **large data files can be stored and accessed** by spark.



amazon
S3

Main Features of Spark

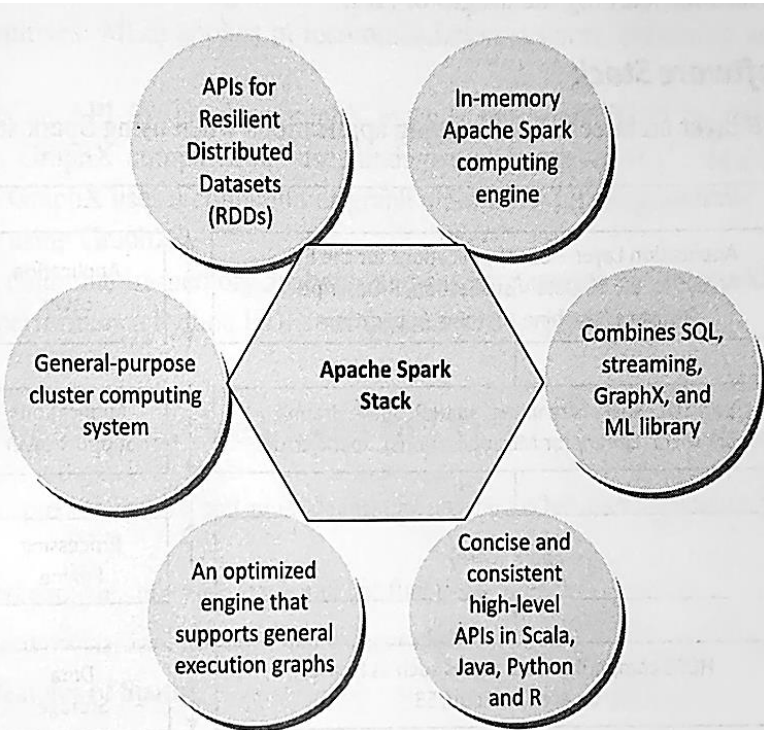


Figure 5.2 Main features of Spark

1. High-Speed Processing

- Spark can **handle complex data very fast**.
- Its **in-memory engine** can be up to **100 times faster than Hadoop** because it **stores data in RAM** instead of reading/writing from disk every time.

2. Uses Both RAM and Disk

Spark **stores temporary results in memory (RAM) first**. If memory is full, it **stores on the disk** — this is **smart memory usage**.

3. Supports Object Store

You can **upload data** from **storage systems like Amazon S3** and **use it directly in Spark**. It treats the **Object Store** as a source of input.

4. High Performance from Cache

If **data is already in memory (cached)**, **Spark runs much faster** because it avoids reading from disk again.

5. RDD Support (Resilient Distributed Datasets)

- RDD is **Spark's core data structure**.
- It **stores data in chunks across different computers** for **parallel processing**.
- You can think of it like a **table** split into different parts.
- RDD's are **fault tolerant**.

6. Handles All Types of Data

- **structured, semi-structured, and unstructured data** from various sources.

7. Supports many new functions in addition to map and reduce functions.

8. **Spark waits before actually running a task** — it **delays execution** until needed to **optimize performance**.

9. Spark has **simple and consistent APIs** in **Scala, Java, and Python**, making it easy for developers to write code. Spark supports **Scala, Java, Python, Clojure, and R**, so many developers can use it.

10. Spark provides an **interactive tool (shell)** for **exploring data quickly using Scala or Python**. It's helpful for **testing code and learning APIs**.

Spark Software stack

Spark SQL – For structured data.

Spark Streaming – For real-time data.

SparkR – Spark interface for R language.

Spark GraphX – For graph processing.

Spark MLlib – For machine learning.

Spark Arrow – For columnar in-memory analytics.

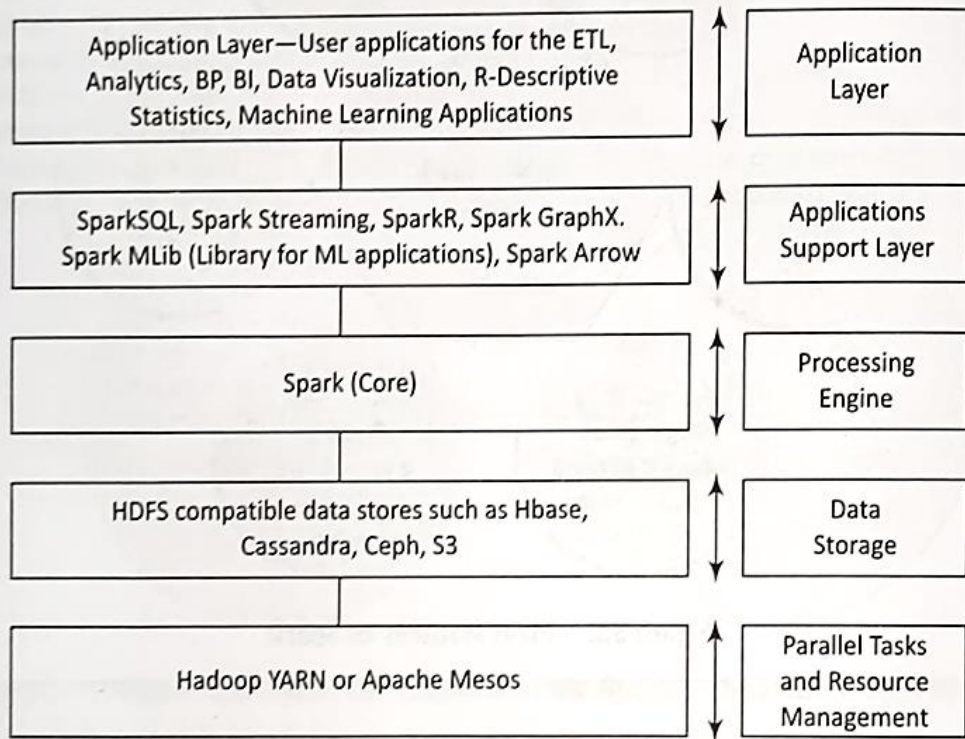


Figure 5.3 Five-layer architecture for running applications using Spark stack

Spark Stack Components

1. Spark SQL

- **Runs SQL queries** on spark data.
- Supports **JDBC, and ODBC**.
- Works with **dataframes, hive, cassandra, parquet, etc.**
- **Allows UDF** (User defined functions).

2. Spark Streaming

- For **real-time streaming data**.
- Uses **Dstream** (series of RDD's)
- **Processes data in micro-batches**.

3. SparkR

- **Lightweight front-end** for spark from R.
- **Uses SparkR API** via RDD class.
- Run jobs from R shell on cluster.

4. Spark Mllib

- **Machine Learning library** in spark.

- It includes **Classification, regression, clustering, collaborative filtering, dimensionality reduction, optimization**.
- It is used in **recommender systems and classification tasks**.

5. Spark GraphX

- **API for graph processing**.
- Extends RDD with **Resilient Distributed Property**.
- Has Operators: **subgraph, joinVertices, aggregateMessages**.
- Has **many graph algorithms**.

6. Spark Arrow

- For **columnar in-memory analytics**.
- Supports **vectorised UDFs (V-UDFs)**
- Used in **high-performance Python UDFs for SerDe** (Serialization/Deserialization) **pipelines**.

Introduction to data analysis with spark

- “Data Analysis is the process of carefully using statistical and/or logical methods to **describe**, **summarize**, and **assess** data.”

Consider a car company that sells five different car models. Each model comes in 16 different colours. The cars are sold through many showrooms across the country.

By analyzing **annual sales** and **profits** for each model, each colour, in each region, and from each showroom, the company can discover useful patterns. This helps them make smart decisions. For example, they can:

- Understand which cars sell best in which areas.
- Plan future production and marketing strategies.

The company also uses **predictive analytics**—this means using past data to predict future trends and decide on future strategies for manufacturing, sales, and customer outreach.

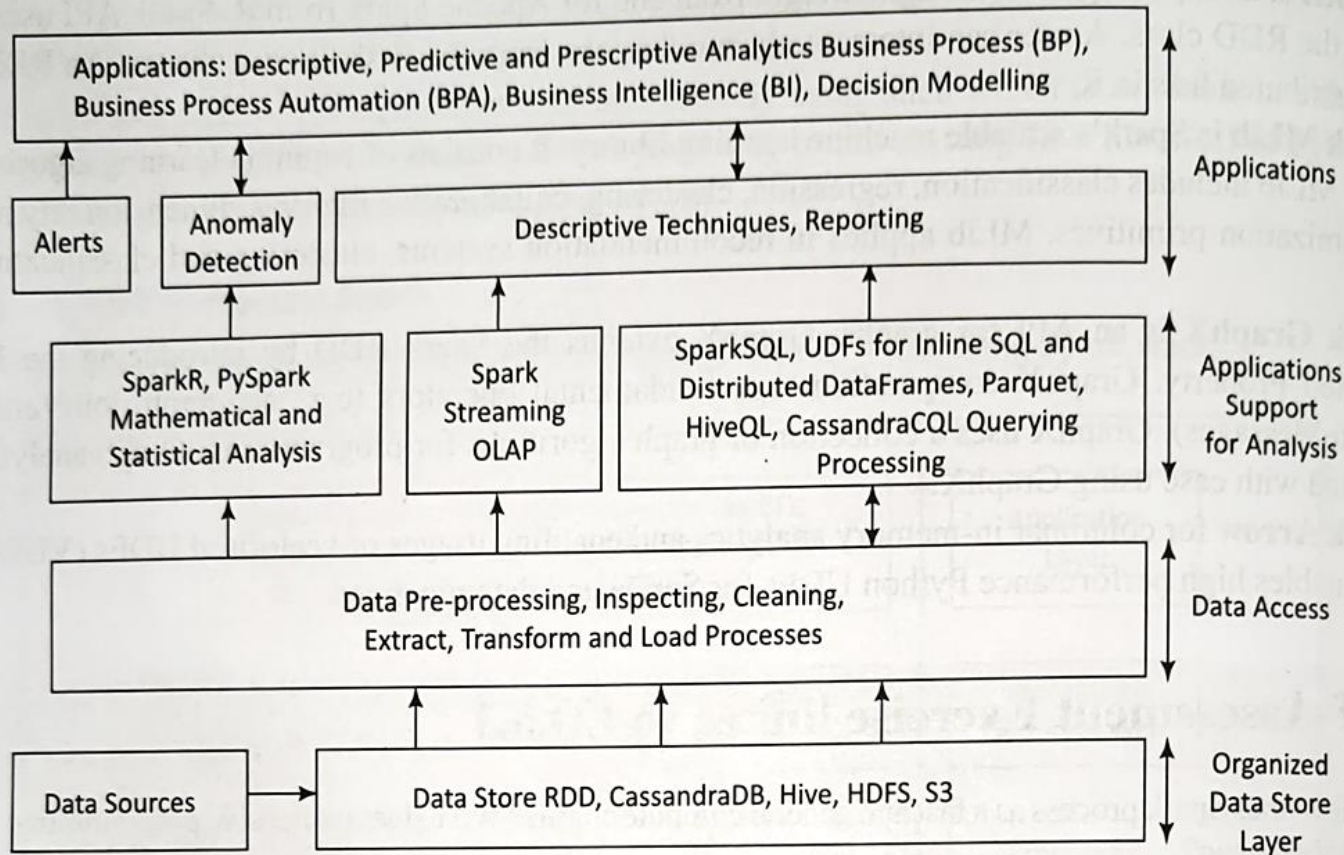


Figure 5.4 Steps between acquisition of data from different sources and its applications

Steps of Data Analysis:

1. Data Storage:

Collect and store data from different sources. Storage can be in systems like HDFS, Cassandra, Hive, or S3.

2. Data Pre-processing:

Clean and prepare the data. This includes:

- Removing incorrect or extreme values.
- Filtering out irrelevant or

repeated information.

- Editing and organizing data for analysis.
- Validating and transforming data.

3. ETL (Extract, Transform, Load):

Extract data from various sources, transform it into the required format, and load it into the system for analysis.

4. Applications of Analyzed Data:

Once the data is analyzed using statistical or logical methods, the results can be used in different ways to support decision-making. These applications help businesses understand the past, predict the future, and decide what actions to take next.

The main types of data analysis applications are:

- **Descriptive Analytics** – *What happened?*
 - This type of analysis looks at past data to explain what has already occurred.
 - Example: A car company checks last year's sales data to see which model sold the most.
- **Predictive Analytics** – *What is likely to happen?*
 - This uses past and current data to predict future trends.
 - Example: Based on current demand and past trends, the company predicts that a certain car model will be in high demand next year.
- **Prescriptive Analytics** – *What actions should be taken?*
 - This suggests possible actions to achieve desired outcomes.
 - Example: The analysis recommends increasing the production of a high-demand model and improving marketing in certain regions.

These analytics support:

- **Business Processes (BP):**
The regular operations and activities of a business (like sales, production, etc.).
- **Business Process Automation (BPA):**
Using technology to automate routine tasks (like auto-generating sales reports).
- **Business Intelligence (BI):**
Using data to help make smart business decisions.
- **Decision Modeling:**
Creating models or plans to help managers choose the best actions.
- **Knowledge Discovery:**
Finding new patterns, trends, or useful information hidden in large sets of data.

**Data Sources → Data Storage → Preprocessing & ETL
→ Querying & Analysis → Reporting & Applications**

Spark SQL

- **Spark SQL** is a **module (part)** of **Apache Spark** that allows you to work with **structured data** using **SQL (Structured Query Language)**.
- It uses special tools like:
 1. **Catalyst Optimizer** – makes SQL queries run faster.
 2. **Tungsten Engine** – improves execution performance.
- Spark SQL will allow to :
 - Run **SQL queries** just like in databases (e.g., SELECT, WHERE, GROUP BY)
 - Work with big data stored in **files (like JSON, CSV, Parquet)** or **databases (like Hive, Cassandra, MySQL)**
 - Use **DataFrames**, which are like tables in memory
- **What Spark SQL Does**
 - It processes structured data such as tables from databases.
 - It provides flexible APIs that allow support for many types of data sources like JSON, Hive, Cassandra, and Parquet.
 - It performs ETL operations, which means it can extract data from different sources, transform it into the required format, and then load it for further processing and analysis. After this, users can run custom SQL queries on the data.

Features of Spark SQL

- Spark SQL supports **many programming languages** including **SparkR, PySpark, Python, and Java**. These can be used for writing code to analyze data.
- It **allows applications to connect to databases using JDBC and ODBC APIs**. This means external programs like Java-based applications or Microsoft Excel can connect to Spark SQL to fetch or analyze data.
- Spark SQL can **read data from different file formats** such as Hive tables, JSON files, and Parquet files. It **can transform this data into a suitable format** and then **allow users to run ad hoc queries**. (An **ad hoc query** is a one-time query designed for a specific purpose. For example, a SQL query to select records based on a certain ID.)
- It includes **support for Hive**, allowing it to **use Hive tables and databases**. It also supports **User Defined Functions (UDFs)**.
- Spark SQL **supports query languages like HiveQL and Cassandra CQL**. This means **users can run queries on data stored in Hive and Cassandra** using their native query languages.
- It **supports Spark Streaming** for **processing both OLTP and structured streaming data**. OLTP stands for Online Transaction Processing, which is useful for real-time applications.

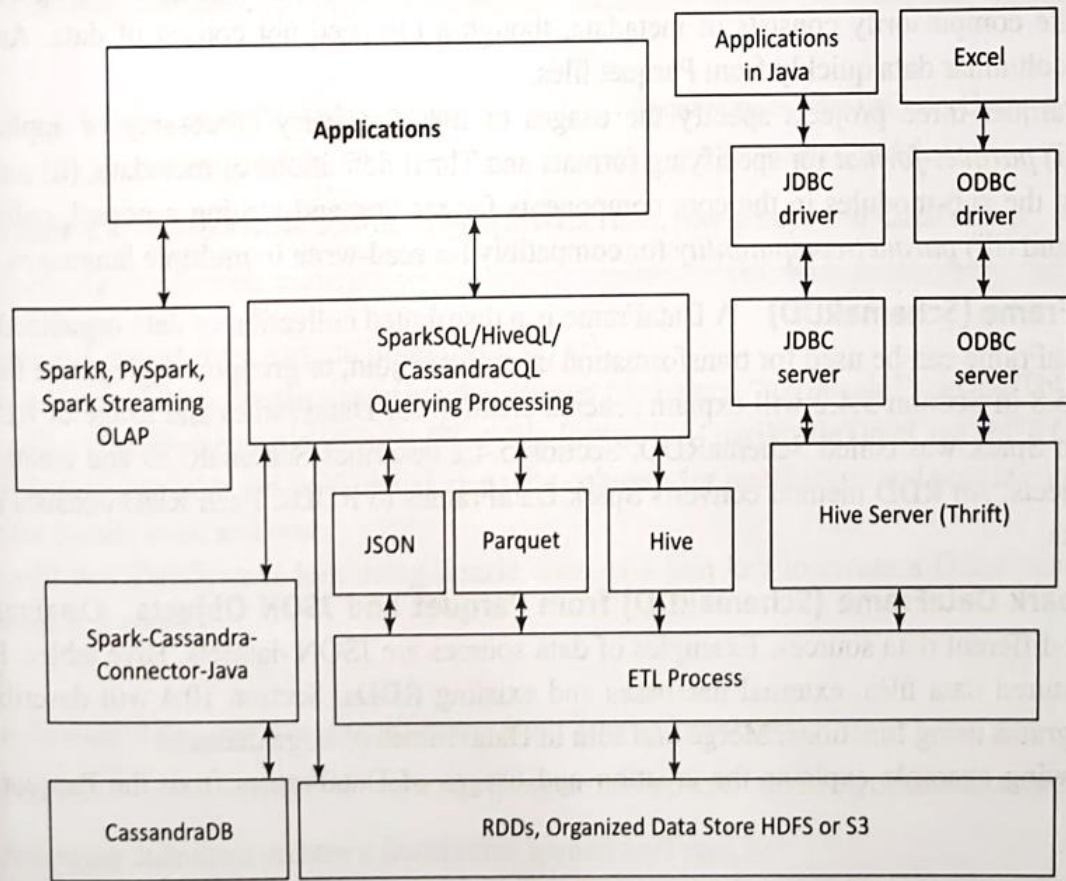


Figure 5.5 Connectivity between the applications and Spark SQL

1. RDD's, HDFS, and S3

- **RDD (Resilient Distributed Dataset):** Spark's internal data structure to store and process data in memory.
- **HDFS :** A distributed file system to store big files (used in big data systems).
- **Amazon S3:** cloud storage by amazon.

2. ETL Process

- **Extract:** Pull data from source (e.g., Cassandra, JSON, Parquet).
- **Transform:** Clean or format the data (e.g., change date format, remove duplicates).
- **Load:** Store the cleaned data back in the required format or structure.

3. JSON, Parquet, Hive:

These are the data formats used to store structured or semistructured data.

- **JSON** – It's a plain text format, easy to read. It is good for web data or logs.
- **Parquet** – a colum-oriented binary format – faster and can be compressed. It is good for analytics and big data processing. Spark reads it faster than JSON.
- **Hive** – a dataware house system on top of HDFS. Data is stored in tables, similar to SQL databases.

4. SparkSQL / HiveQL / CassandraCQL (Query Processing Layer)

This block processes the queries on the data.

- **SparkSQL:** SQL like queries on Spark dataframes.
- **HiveQL :** Hive specific SQL commands.
- **CassandraCQL:** used to query cassandra databases.

Assume data as library, and this layer as a librarian, who knows how to find any book (data) using your search query (SQL).

5. SparkR, PySpark, Spark Streaming, OLAP:

These are **tools used to access and process data using Spark**.

- **PySpark:** Use Python to write Spark code.
- **SparkR:** Use R language for Spark.
- **Spark Streaming:** Process real-time data (like live Twitter feed).
- **OLAP (Online Analytical Processing):** For deep analytics, such as slicing and dicing data for business insights.

6. Spark-Cassandra Connector-Java :

- This allows **Spark to directly connect and fetch data from CassandraDB**, a NoSQL database.

7. CassandraDB:

- A high-performance **NoSQL database**, suitable for handling huge volumes of fast-changing data.

8. Applications:

- General category – this includes any program or platform that wants to use the results from Spark SQL.

Examples:

- Business dashboards
- Reporting tools
- Machine learning pipelines

9. Applications in Java / Excel :

Applications like:

- Java-based web apps
- Microsoft Excel sheets
- These need a way to **connect with Spark SQL** to fetch data.

10. JDBC/ODBC driver

- **JDBC (Java Database Connectivity)** – used by Java applications.
- **ODBC (Open Database Connectivity)** – used by tools like Excel.

These drivers are like **translators** that let Excel or Java apps send SQL queries to Spark.

11. JDBC/ODBC Server

- These are the servers handle incoming queries from JDBC/ODBC drivers.

12. Hive Server (Thrift Server)

The hive server (Thrift server) is a bridge between: external applications (Like excel, Java apps) and SparkSQL or hive for querying big data.

It receives SQL queries from external tools using ODBC or JDBC drivers, sends those queries to sparkSQL or hive, and then returns the results.

Many external tools (like Excel or BI tools) do **not directly understand Spark or Hive**. They communicate using **ODBC or JDBC**, which are standard database connectors.

So, Hive Server (Thrift) acts like a **translator** or **middleman**:

- It **accepts requests** in JDBC/ODBC format.
- It **translates** them to Spark SQL/HiveQL.
- It **sends the results** back to the original application.

Example workflow of the block diagram

Step	Action
1	Excel connects to Spark via ODBC.
2	User uses Excel's Query Wizard or Power Query to select a table or filter data.
3	Excel generates the SQL query in the background, such as: <code>SELECT * FROM sales WHERE month = 'April'</code>
4	The ODBC driver sends this query to the Thrift Server.
5	SparkSQL executes it and sends back the data.
6	Excel displays the result in a worksheet or PivotTable.

What is RDD?	A fault-tolerant, distributed data collection in Spark
Why use it?	Parallel processing, fault recovery, speed
Role in system	Forms the base layer for all Spark processing
When to use it?	When you need control over data or complex operations
Converts to	DataFrames for SQL processing

Using HiveQL for Spark SQL

Spark SQL lets you run **SQL queries** on large data using Apache Spark. It works with data stored in different formats like Hive, JSON, Parquet, etc.

1. SQLContext

- It's used to run SQL queries.
- It **doesn't need Hive** to run.
- Good for simple SQL queries and working with JSON, Parquet, etc.

2. HiveContext

- It's an extended version of SQLContext.
- It **connects with Hive Server**.
- Allows you to run **HiveQL commands**.
- Gives access to **Hive features** like tables, UDFs, and Hive's metadata.
- Use it when you want to access data in Hive

Use of Aggregation and Statistical Functions

Aggregation Functions in HiveQL

Function	Role of it	Example
count(*)	Counts all rows	Total number of records
count(expr)	Counts non-null values in a column	Count of values in "age" column
sum(col)	Adds up values	Total sales
sum(DISTINCT col)	Adds unique values only	Sum of unique sales amounts
avg(col)	Average of a column	Average marks
avg(DISTINCT col)	Average of unique values	Avg of unique scores
min(col)	Minimum value	Lowest price
max(col)	Maximum value	Highest rating

Statistical Functions in HiveQL

Function	Role of it	Example
stdev()	Standard deviation	How spread out the data is
sampleStdev()	Sample standard deviation	Same as above, but for sample data
variance()	Variance of values	How much data varies
sampleVariance()	Sample variance	Like above, but on sample

Using Python Advanced Features with Spark SQL

Python is a popular programming language known for being:

- **Easy to read and write**
- **High-level** (you don't worry about system-level things)
- **Object-Oriented** (you can use classes and objects)
- **Interactive** (can run code line by line)

It supports:

- **Basic data types:** like numbers, strings, booleans
- **Data containers:** like lists, dictionaries (`{}`), sets, and tuples
- **Functions and classes:** you can write reusable code and create objects

Python comes with many **built-in libraries** for different tasks:

Purpose	Examples
Regular expressions	re module
Unit testing	unittest
Web work	cgi, http, urllib
Image handling	PIL, imageio
Email	smtplib, email
Threading	threading
Databases	sqlite3, mysql-connector

Python + SparkSQL = PySpark

- PySpark is the python API for apache spark, means spark programs can be written in python.
- PySpark is more powerful because
 - Processing of big data can be done using Spark SQL.
 - Use python's simple syntax and powerful libraries.
 - Work easily with dataframes, hive tables, JSON, etc.

```
from pyspark.sql import SparkSession

# Start a Spark session
spark = SparkSession.builder.appName("Example").getOrCreate()

# Load data as DataFrame
df = spark.read.json("data.json")

# Run SQL-Like query
df.createOrReplaceTempView("people")
result = spark.sql("SELECT name, age FROM people WHERE age > 30")

# Show result
result.show()
```

Python Libraries for Analysis

- NumPy and SciPy are python libraries, where NumPy – Numerical Python, and SciPy – Scientific Python. They help in doing math, **statistics, and scientific calculations** quickly.
- They are **free** and **open-source**, and widely used in **data analysis** and **machine learning**.
- Other useful python libraries for data analysis are as follows:

Library	Use
NumPy	Fast math operations, arrays, statistics
SciPy	Advanced scientific computations
Scikit-learn	Machine learning models
Pandas	Handling data in tables (like Excel)
StatsModels	Statistics and regression analysis
Matplotlib	Drawing charts and graphs

About spark and python's UDF's

- UDF – User Defined Function, which is written to process each line of data.
- Problem with old UDF's :- Spark had to **convert data** between Python and JVM (Java Virtual Machine). This **conversion (called SerDe)** was slow (Serialization and Deserialization).
- Spark 2.3, Introduced **Vectorized UDFs** (VUDFs) using **Apache Arrow**. This allows data to move faster between Spark and Python — **improving performance**. Now, Python UDFs can work with **entire columns (vectors)** instead of **one row at a time**.

NumPy

NumPy is like a **super-fast calculator** for handling arrays (tables of numbers).

NumPy Features

Feature	Functionality	Example
N-dimensional arrays	Stores data in 1D, 2D, 3D... formats	array([[1,2],[3,4]])
Math operations	Do math on arrays	np.sqrt(arr)
Linear Algebra	Matrix multiplication, inversion	np.dot(A, B)
Random Numbers	Create random data	np.random.rand(3,2)
Fourier Transforms	Signal processing	np.fft.fft()
Array indexing	Select part of the array	arr[0:2]
Broadcasting	Apply operations across shapes	arr + 10
Integration with C/C++	Use C for speed boost	np.ctypeslib

- NumPy can do many array operations:
 - **Creating arrays:** `np.array([1,2,3])`
 - **Sorting:** `np.sort(arr)`
 - **Reshaping:** `arr.reshape(2,3)`
 - **Math functions:** `np.sin(x)`, `np.log(x)`, `np.ceil(x)`
 - **Statistics:** `np.mean(arr)`, `np.std(arr)`
- NumPy allows:
 - **Import/export of data** (save/load)
 - **Interfacing with databases**
 - **Reading and writing from files**

Table 5.1 Examples of NumPy functions for data analysis problems

Function	Description	Function	Description
<code>np.loadtxt('file.txt')</code>	Loads a text file	<code>np.mean(arr, axis = 0)</code>	Returns mean along a specific axis
<code>np.genfromtxt('file.csv', delimiter= ',')</code>	Loads a csv file with comma as the delimiter between records	<code>np.sum(); np.min(),</code>	Returns the sum and minimum of the array
<code>np.savetxt('file.txt,' arr, delimiter= ' ')</code>	Saves a text file which is an array of strings separated by a space each.	<code>np.max(arr, axis = 0)</code>	Returns the maximum along a specific axis
<code>np.genfromtxt('file.csv', arr, delimiter= ',')</code>	Saves a CSV file which is an array of strings separated by a comma each.	<code>np.var(arr)</code>	Returns the variance of array
<code>arr.sort()</code>	Sorts an array	<code>np.std(arr, axis = 0)</code>	Returns the standard deviation of a specific axis
<code>np.add (arr1, arr2)</code>	Performs a vector addition of array 1 and array 2	<code>np.corr()</code>	Returns the correlation coefficient

SciPy

- **SciPy** is a **Python** library built **on top of NumPy**.
- It adds **more advanced scientific functions** for working with numbers, equations, and data.
- SciPy = NumPy + Extra scientific tools

Feature	Explanation
Built on NumPy	Uses NumPy's arrays and functions as a base
Reads MATLAB files	Can open .mat files used in MATLAB
Special functions	Includes tools for math operations like integration and optimization
Distance functions	Can calculate distances between points (e.g., for clustering or geometry)
Mesh grids	Helps create 2D/3D grids for simulations or plotting
Shape tools	Resize and reshape arrays
Polynomial functions	Work with polynomials (like equations in x)
Real and imaginary support	Can handle complex numbers like $3 + 4j$
Data type casting	Change data types (e.g., int $\rightarrow$ float) easily
Matrix support	Create and work with matrices (used in math, ML, etc.)
Works with Spark	Can be used along with Spark for advanced matrix operations

Table 5.2 Examples of SciPy functions for data analysis problems

Function	Description	Function	Description
np.c_[b, c]	Create Stacked column-wise array	np.cast ['f'] (np.pi)	Casts an object into a data type
b.flatten()	Flattens the array	from numpy import polyID p = polyID ([2, 3, 4])	Creates a polynomial object p
np.vsplit (c, 2) and np.hsplit (d, 2)	Functions for vertically splitting and horizontally splitting the array at the end of the second index	A.I, A.T, A.H	Inverses, transposes, and conjugate transposes the matrix, A

Function	Description	Function	Description
linalg.det(A)	Returns the determinate of A	np.select ([c<4], [c*2])	Returns values from a list of arrays depending on the conditions
np.img(c)	Returns the imaginary part of the array elements	A = np.matrix ([3, 4], [5, 6])	Creates a matrix

Pandas

- **Pandas** is a **Python library** used for **data analysis** and **manipulation**.
- The name “Pandas” come from:
 - a) Pan = Panel data (used in economics)
 - b) da = Data
- Python has 3 main data structures:

Structure	Explanation	Example
Series	1D labeled data (like a column)	<code>pd.Series([1, 2, 3])</code>
DataFrame	2D table (like Excel)	<code>pd.DataFrame({'A': [1, 2], 'B': [3, 4]})</code>
Panel (older)	3D container of DataFrames	Used for panel data, but now less common

- A **Panel** is like a **collection of DataFrames**.
- About Axis in panel :

Axis	Meaning
Items (axis 0)	The DataFrame labels (like 'Sheet1', 'Sheet2')
Major Axis (axis 1)	Rows in each DataFrame
Minor Axis (axis 2)	Columns in each DataFrame

Other features of pandas are:

1. SQL-like Operations

You can use:

- SELECT: `df[['column1']]`
- WHERE: `df[df['age'] > 30]`
- GROUPBY: `df.groupby('region').sum()`
- JOIN: `pd.merge(df1, df2, on='id')`

2. GroupBy (Split-Apply-Combine)

- Break data into groups → process each group → combine result.

```
df.groupby('category').sum()
```

3. Size Mutability

- You can **add or delete columns** in DataFrames anytime.

```
df['new_col'] = df['old_col'] * 2      # Add column
```

```
del df['old_col']                     # Delete column
```

4. Merging and Concatenation

- You can **merge** DataFrames like SQL joins, or **combine** them vertically/horizontally.

```
pd.concat([df1, df2])          # Stack vertically
```

5. Slicing and Dicing

You can select **rows**, **columns**, or **blocks** of data:

```
df.loc['2023']    # Row
```

```
df['sales']       # Column
```

```
df.iloc[0:2, 0:2] # Specific block
```

6. Pandas package includes Rpy interface for R functions plus additional functions.

7. Panda ecosystem has statics, machine learning, integrated development environment (IDE), API and several out of core features.

User Defined Functions

- A **UDF** is a custom function **you write yourself** to perform a specific task on your data.
- It helps when the **built-in Spark functions** are not enough.
- UDFs can be used inside **SQL queries** or **DataFrame operations**.

Disadvantages of older UDF

- **Standard UDFs** take **one row at a time** → This is **slow**.
- Every time Python and Spark exchange data, there is an **overhead** due to:
 - **SerDe** = Serialization and Deserialization (converting data between Python and Spark JVM).
- Earlier: Most UDFs were written in **Java/Scala**, and Python had to **call** them, which added complexity.

Solution for above Disadvantage

- PySpark allows you to write UDFs directly in Python. You can register them and call them inside SQL or apply them on DataFrames.

```
from pyspark.sql.functions import udf
from pyspark.sql.types import IntegerType

# Your custom function
def max_sale(sales_list):
    return max(sales_list)

# Register it as a UDF
spark.udf.register("maxSale", max_sale, IntegerType())

# Use it in SQL or DataFrame
df.createOrReplaceTempView("sales")
spark.sql("SELECT showroom_id, maxSale(sales) FROM sales").show()
```

Why use UDF's

- Reusable logic
- Shorter code
- Useful when built-in spark SQL functions cant solve your problem.
- You can apply them to different products or datasets (eg., cars, toys, electronics)

Vectorized UDF (VUDF)

- **Vectorized UDFs** are faster and smarter versions of UDFs.
- They use **Pandas Series** instead of single rows — meaning **batch/column-wise operations**.
- They avoid unnecessary overhead by using **Apache Arrow** (a high-performance library for memory sharing between Python and JVM).

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import pandas_udf
from pyspark.sql.types import DoubleType
import pandas as pd

# 1. Create Spark session
spark = SparkSession.builder.appName("VUDF Example").getOrCreate()

# 2. Sample data
data = [
    ("Alice", 50000),
    ("Bob", 60000),
    ("Charlie", 70000),
    ("David", 80000)
]

# 3. Create DataFrame
df = spark.createDataFrame(data, ["name", "salary"])

# 4. Define a vectorized UDF to calculate bonus (10% of salary)
@pandas_udf(DoubleType())
def calculate_bonus(salaries: pd.Series) -> pd.Series:
    return salaries * 0.10 # 10% bonus

# 5. Use the VUDF to add a new column
df_with_bonus = df.withColumn("bonus", calculate_bonus(df["salary"]))

# 6. Show the result
df_with_bonus.show()
```

Benefits of VUDF:

- Faster than regular UDF
- Works like Pandas (easy to write and read)
- Uses **columnar memory**, which is faster for Big Data

What is Apache Arrow?

- It's a tool that helps Spark and Python exchange data **quickly**.
- Used in VUDFs to **reduce conversion time** (serialization/deserialization).

Grouped Vectorized UDFs (GVUDFs)

- GVUDFs work **on groups of data** — for example, "group by showroom ID, then calculate total yearly sales."
- Uses Pandas' **split-apply-combine** logic.

Split-Apply-Combine Steps:

- **Split** the DataFrame into groups (e.g., group by showroom ID)
- **Apply** a custom function on each group
- **Combine** the results back into one DataFrame

```
from pyspark.sql.functions import pandas_udf
from pyspark.sql.types import StructType, StructField, StringType, DoubleType
import pandas as pd
```

Define schema of output

```
schema = StructType([
    StructField("showroom_id", StringType()),
    StructField("total_sales", DoubleType())
])
```

Define grouped UDF

```
@pandas_udf(schema, functionType="grouped_map")
def total_sales_per_showroom(pdf: pd.DataFrame) -> pd.DataFrame:
    total = pdf['sales'].sum()
    showroom_id = pdf['showroom_id'].iloc[0]
    return pd.DataFrame([[showroom_id, total]], columns=["showroom_id", "total_sales"])
```

Apply it

```
df.groupBy("showroom_id").apply(total_sales_per_showroom).show()
```

Comparison of VUDF (Vectorized UDF) and GVUDF (Grouped Vectorized UDF)

Term	Description
VUDF (Vectorized UDF)	A function that processes one column (Series) of a DataFrame at a time.
GVUDF (Grouped Vectorized UDF)	A function that processes groups of rows (DataFrames) together, based on a groupBy.

Type	Input
VUDF	pandas.Series (one column)
GVUDF	pandas.DataFrame (entire group of rows)

Use When You Need	Use
To apply a custom function to a single column (like salary → bonus)	VUDF
To apply a function on each group (like per showroom, per year, etc.)	GVUDF

UDF Type	Function Decorator
VUDF	@pandas_udf(return_type)
GVUDF	@pandas_udf(schema, functionType="grouped_map")

UDF Type	Output
VUDF	pandas.Series (column)
GVUDF	pandas.DataFrame (multiple columns and rows)

UDF Type	Example
VUDF	Calculate 10% bonus of each employee's salary
GVUDF	Compute total sales per showroom using groupBy on showroom_id

MODULE 5

Text Mining

Text mining is the art and science of discovering knowledge, insights and patterns from an organized collection of textual databases. Textual mining can help with frequency analysis of important terms, and their semantic relationships.

Text is an important part of the growing data in the world. Social media technologies have enabled users to become producers of text and images and other kinds of information. Text mining can be applied to large-scale social media data for gathering preferences, and measuring emotional sentiments. It can also be applied to societal, organizational and individual scales.

Text Mining Applications

Text mining is a useful tool in the hands of chief knowledge officers to extract knowledge relevant to an organization. Text mining can be used across industry sectors and application areas, including decision support, sentiment analysis, fraud detection, survey analysis, and many more.

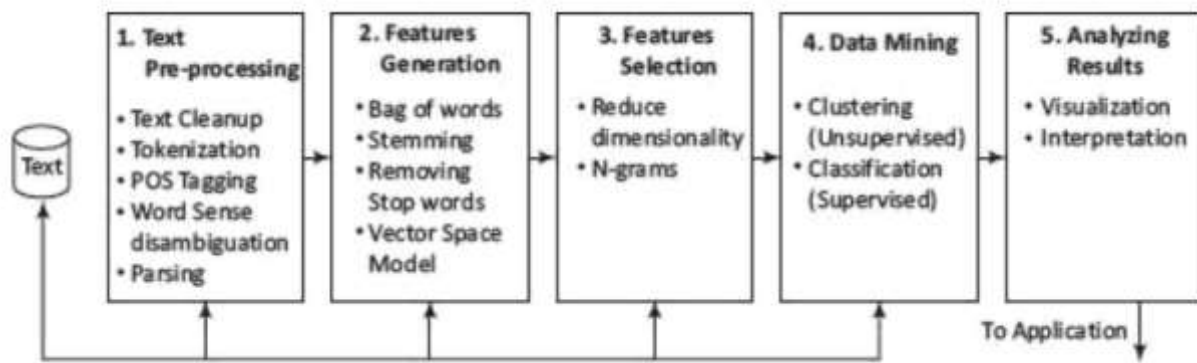
1. *Marketing:* The voice of the customer can be captured in its native and raw format and then analyzed for customer preferences and complaints.
 1. Social personas are a clustering technique to develop customer segments of interest. Consumer input from social media sources, such as reviews, blogs, and tweets, contain numerous leading indicators that can be used towards anticipating and predicting consumer behavior.
 2. A 'listening platform' is a text mining application, that in real time, gathers social media, blogs, and other textual feedback, and filters out the chatter to extract true consumer sentiment. The insights can lead to more effective product marketing and better customer service.
2. The customer call center conversations and records can be analyzed for patterns of customer complaints. Decision trees can organize this data to create decision choices

that could help with product management activities and to become proactive in avoiding those complaints.

3. *Business operations:* Many aspects of business functioning can be accurately gauged from analyzing text./
 1. Social network analysis and text mining can be applied to emails, blogs, social media and other data to measure the emotional states and the mood of employee populations. Sentiment analysis can reveal early signs of employee dissatisfaction which can then can be proactively managed.
 2. Studying people as emotional investors and using text analysis of the social Internet to measure mass psychology can help in obtaining superior investment returns.
3. *Legal:* In legal applications, lawyers and paralegals can more easily search case histories and laws for relevant documents in a particular case to improve their chances of winning.
 1. Text mining is also embedded in e-discovery platforms that help in minimizing risk in the process of sharing legally mandated documents.
 2. Case histories, testimonies, and client meeting notes can reveal additional information, such as morbidities in a healthcare situation that can help better predict high-cost injuries and prevent costs.
4. *Governance and Politics:* Governments can be overturned based on a tweet originating from a self-immolating fruit-vendor in Tunisia.
 1. Social network analysis and text mining of large-scale social media data can be used for measuring the emotional states and the mood of constituent populations. Micro-targeting constituents with specific messages gleaned from social media analysis can be a more efficient use of resources when fighting democratic elections.
 2. In geopolitical security, internet chatter can be processed for real- time information and to connect the dots on any emerging threats.
 3. In academic, research streams could be meta-analyzed for underlying research trends.

Text Mining Process

Text Mining is a rapidly evolving area of research. As the amount of social media and other text data grows, there is need for efficient abstraction and categorization of meaningful information from the text.



The five phases for processing text are as follows:

Phase 1: Text pre-processing enables Syntactic/Semantic text-analysis and does the followings:

1. *Text cleanup* is a process of removing unnecessary or unwanted information. Text cleanup converts the raw data by filling up the missing values, identifies and removes outliers, and resolves the inconsistencies. For example, removing comments, removing or escaping "%20" from URL for the web pages or cleanup the typing error, such as teh (the), do n't (do not) [%20 specifies space in a URL].
2. *Tokenization* is a process of splitting the cleanup text into tokens (words) using white spaces and punctuation marks as delimiters.
3. *Part of Speech (POS) tagging* is a method that attempts labeling of each token (word) with an appropriate POS. Tagging helps in recognizing names of people, places, organizations and titles. English language set includes the noun, verb, adverb, adjective, prepositions and conjunctions. Part of Speech encoded in the annotation system of the Penn Treebank Project has 36 POS tags.⁴
4. *Word sense disambiguation* is a method, which identifies the sense of a word used in a sentence; that gives meaning in case the word has multiple meanings. The methods, which resolve the ambiguity of words can be context or proximity based. Some examples of such

words are bear, bank, cell and bass.

5. *Parsing* is a method, which generates a parse-tree for each sentence. Parsing attempts and infers the precise grammatical relationships between different words in a given sentence.

Phase 2: Features Generation is a process which first defines features (variables, predictors). Some of the ways of feature generations are:

1. *Bag of words*-Order of words is not that important for certain applications.

Text document is represented by the words it contains (and their occurrences). Document classification methods commonly use the bag-of-words model. The pre-processing of a document first provides a document with a bag of words. Document classification methods then use the occurrence (frequency) of each word as a feature for training a classifier. Algorithms do not directly apply on the bag of words, but use the frequencies.

2. *Stemming*-identifies a word by its root.

(i) Normalizes or unifies variations of the same concept, such as *speak* for three variations, i.e., speaking, speaks, speakers denoted by [speaking, speaks, speaker-+ speak]

(ii) Removes plurals, normalizes verb tenses and remove affixes.

Stemming reduces the word to its most basic element. For example, impurification -+ pure.

3. *Removing stop words* from the feature space-they are the common words, unlikely to help text mining. The search program tries to ignore stop words. For example, ignores *a, at, for, it, in* and *are*.

4. *Vector Space Model (VSM)*-is an algebraic model for representing text documents as vector of identifiers, word frequencies or terms in the document index. VSM uses the method of term frequency-inverse document frequency (TF-IDF) and evaluates how important is a word in a document.

When used in document classification, VSM also refers to the bag-of-words model. This bag of words is required to be converted into a term-vector in VSM. The term vector provides the numeric values corresponding to each term appearing in a document. The term vector is very helpful in feature generation and selection.

Term frequency and inverse document frequency (IDF) are important metrics in text analysis. TF-IDF weighting is most common- Instead of the simple TF, IDF is used to weight the importance of word in the document.

Phase 3: Features Selection is the process that selects a subset of features by rejecting irrelevant and/or redundant features (variables, predictors or dimension) according to defined criteria. Feature selection process does the following:

1. *Dimensionality reduction*-Feature selection is one of the methods of division and therefore, dimension reduction. The basic objective is to eliminate irrelevant and redundant data. Redundant features are those, which provide no extra information. Irrelevant features provide no useful or relevant information in any context.

Principal Component Analysis (PCA) and Linear Discriminate Analysis (LDA) are dimension reduction methods. Discrimination ability of a feature measures relevancy of features. Correlation helps in finding the redundancy of the feature. Two features are redundant to each other if their values correlate with each other.

2. *N-gram evaluation*-finding the number of consecutive words of interest and extract them. For example, 2-gram is a two words sequence, ["tasty food", "Good one"]. 3-gram is a three words sequence, ["Crime Investigation Department"].
3. *Noise detection and evaluation of outliers* methods do the identification of unusual or suspicious items, events or observations from the data set. This step helps in cleaning the data.

The feature selection algorithm reduces dimensionality that not only improves the performance of learning algorithm but also reduces the storage requirement for a dataset. The process enhances data understanding and its visualization.

Phase 4: Data mining techniques enable insights about the structured database that resulted from the previous phases. Examples of techniques are:

1. Unsupervised learning (for example, clustering)

(i) The class labels (categories) of training data are unknown

(ii) Establish the existence of groups or clusters in the data

Good clustering methods use high intra-cluster similarity and low inter-cluster similarity.

Examples of uses - biogs, pattern

and trends.

2. *Supervised learning (for example, classification)*

(i) The training data is labeled indicating the class

(ii) New data is classified based on the training set

Classification is correct when the known label of test sample is identical with the resulting class computed from the classification model.

Examples of uses are *news filtering application*, where it is required to automatically assign incoming documents to pre-defined categories; *email spam filtering*, where it is identified whether incoming email messages are spam or not.

Example of text classification methods are *Naive Bayes Classifier* and *SVMs*.

3. *Identifying evolutionary patterns* in temporal text streams-the method is useful in a wide range of applications, such as summarizing of events in news articles and extracting the research trends in the scientific literature.

Phase 5: Analysing results

- (i) Evaluate the outcome of the complete process.
- (ii) Interpretation of Result- If acceptable then results obtained can be used as an input for next set of sequences. Else, the result can be discarded, and try to understand what and why the process failed.
- (iii) Visualization - Prepare visuals from data, and build a prototype.
- (iv) Use the results for further improvement in activities at the enterprise, industry or institution.

Text Mining Challenges

The challenges in the area of text mining can be classified on the basis of documents area-characteristics. Some of the classifications are as follows:

1. NLP issues:

- (i) POS Tagging
- (ii) Ambiguity
- (iii) Tokenization
- (iv) Parsing
- (v) Stemming
- (vi) Synonymy and polysemy

2. Mining techniques:

- (i) Identification of the suitable algorithm(s)
- (ii) Massive amount of data and annotated corpora
- (iii) Concepts and semantic relations extraction
- (iv) When no training data is available

3. Variety of data:

- (i) Different data sources require different approaches and different areas of expertise
- (ii) Unstructured and language independency

4. Information visualization

5. Efficiency when processing real-time text stream

6. Scalability

The different approaches for supervised text classification are:

- (i) K-Nearest Neighbour Method**
- (ii) Support Vector Machine**
- (iii) Naïve Bayes Method**
- (iv) Decision Tree**
- (v) Decision Rule**

Term Document Matrix

This is the heart of the structuring process. Free flowing text can be transformed into numeric data in a TDM, which can then be mined using regular data mining techniques.

1. There are several efficient techniques for identifying key terms from a text. There are less efficient techniques available for creating topics out of them. For the purpose of this discussion, one could call key words, phrases or topics as a term of interest. This approach measures the frequencies of select important terms occurring in each document. This creates a $t \times d$ Term-by-Document Matrix (TDM) where t is the number of terms and d is the number of documents (Table 11.1).
2. Creating a TDM requires making choices of which terms to include. The terms chosen

should reflect the stated purpose of the text mining exercise. The list of terms should be as extensive as needed, but should not include unnecessary stuff that will serve to confuse the analysis, or slow the computation.

	Term-Document Matrix				
Document/ Terms	Investment	Profit	Happy	Success	...
Doc 1	10	4	3	4	
Doc 2	7	2	2		
Doc 3			2	6	
Doc 4	1	5	3		
Doc 5		6		2	
Doc 6	4		2		
...					

Table 1.1: Term-Document Matrix

Here are some considerations in creating a TDM.

1. A large collection of documents mapped to a large bag of words will likely lead to a very sparse matrix if they have few common words. Reducing dimensionality of data will help improve the speed of analysis and meaningfulness of the results. Synonyms, or terms with similar meaning, should be combined and should be counted together, as a common term. This would help reduce the number of distinct terms or words or 'tokens'.
2. Data should be cleaned for spelling errors. Common spelling errors should be ignored and the terms should be combined. Uppercase- lowercase terms should also be combined.
3. When many variants of the same term are used, just the stem of the word would be used to reduce the number of terms. For instance, terms like customer order, ordering, order data, should be combined into a single token word, called 'Order'.
4. On the other side, homonyms (terms with the same spelling but different meanings) should be counted separately. This would enhance the quality of analysis. For example, the term order can mean a customer order, or the ranking of certain choices. These two

should be treated separately. “The boss ordered that the customer orders data analysis be presented in chronological order’. This statement shows three different meanings for the word ‘order’. Thus, there will be a need for a manual review of the TD matrix.

5. Terms with very few occurrences in very few documents should be eliminated from the matrix. This would help increase the density of the matrix and the quality of analysis.
6. The measures in each cell of the matrix could be one of several possibilities. It could be a simple count of the number of occurrences of each term in a document. It could also be the log of that number. It could be the fraction number computed by dividing the frequency count by the total number of words in the document. Or there may be binary values in the matrix to represent whether a term is mentioned or not. The choice of value in the cells will depend upon the purpose of the textanalysis.

At the end of this analysis and cleansing, a well-formed, densely populated, rectangular, TDM will be ready for analysis. The TDM could be mined using all the available data mining techniques.

Mining the TDM

The TDM can be mined to extract patterns/knowledge. A variety of techniques could be applied to the TDM to extract new knowledge.

Predictors of desirable terms could be discovered through predictive techniques, such as regression analysis. Suppose the word profit is a desirable word in a document. The number of occurrences of the word profit in a document could be regressed against many other terms in the TDM. The relative strengths of the coefficients of various predictor variables would show the relative impact of those terms on creating a profitdiscussion.

Predicting the chances of a document being liked is another form of analysis. For example, important speeches made by the CEO or the CFO to investors could be evaluated for quality. If the classification of those documents (such as good or poor speeches) was available, then the terms of TDM could be used to predict the speech class. A decision tree could be constructed that makes a simple tree with a few decision points that predicts the success of a speech 80 percent of the time. This tree could be trained with more data to become better over time.

Clustering techniques can help categorize documents by common profile. For example, documents containing the words investment and profit more often could be bundled together. Similarly, documents containing the words, customer orders and marketing, more often could be bundled together. Thus, a few strongly demarcated bundles could capture the essence of the entire TDM. These bundles could thus help with further processing, such as handing over select documents to others for legal discovery.

Association rule analysis could show relationships of coexistence. Thus, one could say that the words, tasty and sweet, occur together often (say 5 percent of the time); and further, when these two words are present, 70 percent of the time, the word happy, is also present in the document.

Comparing Text Mining and Data Mining

Text Mining is a form of data mining. There are many common elements between Text and Data Mining. However, there are some key differences (Table 1.2). The key difference is that text mining requires conversion of text data into frequency data, before data mining techniques can be applied.

Dimension	Text Mining	Data Mining
Nature of data	Unstructured data: Words, phrases, sentences	Numbers; alphabetical and logical values
Language used	Many languages and dialects used in the world; many languages are extinct, new documents are discovered	Similar numerical systems across the world
Clarity and precision	Sentences can be ambiguous; sentiment may contradict the words	Numbers are precise.

Consistency	Different parts of the text can contradict each other	Different parts of data can be inconsistent, thus, requiring statistical significance analysis
Sentiment	Text may present a clear and consistent or mixed sentiment, across a continuum. Spoken words adds further sentiment	Not applicable
Quality	Spelling errors. Differing values of proper nouns such as names. Varying quality of language translation	Issues with missing values, outliers, etc
Nature of Analysis	Keyword based search; co-existence of themes; Sentiment Mining	A full wide range of statistical and machine learning analysis for relationship and differences

Table 1.2: Comparing Text Mining and Data Mining

Text Mining Best Practices

Many of the best practices that apply to the use of data mining techniques will also apply to text mining.

1. The first and most important practice is to ask the right question. A good question is one which gives an answer and would lead to large payoffs for the organization. The purpose and the key question will define how and at what levels of granularity the TDM would be made. For example, TDM defined for simpler searches would be different from those used for complex semantic analysis or network analysis.
2. A second important practice is to be creative and open in proposing imaginative hypotheses for the solution. Thinking outside the box is important, both in the quality of the proposed solution as well as in finding the high quality data sets required to test the hypothesized solution. For example, a TDM of consumer sentiment data should be combined with customer order data in order to develop a comprehensive view of

customer behavior. It's important to assemble a team that has a healthy mix of technical and business skills.

3. Another important element is to pursue the problem iteratively. Too much data can overwhelm the infrastructure and also befuddle the mind. It is better to divide and conquer the problem with a simpler TDM, with fewer terms and fewer documents and data sources. Expand as needed, in an iterative sequence of steps. In the future, add new terms to help improve predictive accuracy.
4. A variety of data mining tools should be used to test the relationships in the TDM. Different decision tree algorithms could be run alongside cluster analysis and other techniques. Triangulating the findings with multiple techniques, and many what-if scenarios, helps build confidence in the solution. Test the solution in many ways before committing to deploy it.

Naïve Bayes Analysis

Naïve Bayes technique is a supervised machine learning technique that uses probability theory based analysis.

It is machine learning technique that computes the probabilities of an instance of belonging to each of many target classes, given the prior probabilities of classification using individual factors.

The diagram illustrates the Naïve Bayes formula: $P(c|x) = \frac{P(x|c)P(c)}{P(x)}$. Arrows point from the terms in the formula to their respective labels: $P(c|x)$ is labeled 'Posterior Probability', $P(x|c)$ is labeled 'Likelihood', $P(c)$ is labeled 'Class Prior Probability', and $P(x)$ is labeled 'Predictor Prior Probability'. Below the main formula, the expanded formula is shown: $P(c|X) = P(x_1|c) \times P(x_2|c) \times \dots \times P(x_n|c) \times P(c)$.

□ $P(c|x)$ is the posterior probability of class (c, target) given predictor (x, attributes).

- $P(c)$ is the prior probability of class.
- $P(x|c)$ is the likelihood which is the probability of predictor given class.
- $P(x)$ is the prior probability of predictor.

3.1 Probability

- The Bayes Rule provides the formula for the probability of Y given X. But, in real-world problems, you typically have multiple X variables.
- When the features are independent, we can extend the Bayes Rule to what is called Naive Bayes.
- It is called 'Naive' because of the naive assumption that the X's are independent of each other. Regardless of its name, it's a powerful formula.

When there are multiple X variables, we simplify it by *assuming the X's are independent*, so the **Bayes** rule

$$P(Y=k | X) = \frac{P(X | Y=k) * P(Y=k)}{P(X)}$$

where, k is a class of Y

becomes, **Naive Bayes**

$$P(Y=k | X_1..X_n) = \frac{P(X_1 | Y=k) * P(X_2 | Y=k) ... * P(X_n | Y=k) * P(Y=k)}{P(X_1) * P(X_2) ... * P(X_n)}$$

$$P(Y=k | X_1..X_n) = \frac{P(X_1 | Y=k) * P(X_2 | Y=k) ... * P(X_n | Y=k) * P(Y=k)}{P(X_1) * P(X_2) ... * P(X_n)}$$

can be understood as ..

$$\text{Probability of Outcome | Evidence (Posterior Probability)} = \frac{\text{Probability of Likelihood of evidence} * \text{Prior}}{\text{Probability of Evidence}}$$

Probability of Evidence is same for all classes of Y

- In technical jargon, the left-hand-side (LHS) of the equation is understood as the posterior probability or simply the posterior.
- The RHS has 2 terms in the numerator.
- The first term is called the ‘Likelihood of Evidence’. It is nothing but the conditional probability of each X’s given Y is of particular class ‘c’.
- Since all the X’s are assumed to be independent of each other, you can just multiply the ‘likelihoods’ of all the X’s and called it the ‘Probability of likelihood of evidence’. This is known from the training dataset by filtering records where Y=c.
- The second term is called the prior which is the overall probability of Y=c, where c is a class of Y. In simpler terms, Prior = count(Y=c) / n_Records.
- An example is better than an hour of theory. So let’s see one

Naive Bayes Example

- Say you have 1000 fruits which could be either 'banana', 'orange' or 'other'.
- These are the 3 possible classes of the Y variable.
- We have data for the following X variables, all of which are binary (1 or 0).
- Long
- Sweet
- Yellow
- The first few rows of the training dataset look like this:

Fruit	Long (x1)	Sweet (x2)	Yellow (x3)
Orange	0	1	0
Banana	1	0	1
Banana	1	1	1
Other	1	1	0

- For the sake of computing the probabilities, let's aggregate the training data to form a counts table like this.

Type	Long	Not Long	Sweet	Not Sweet	Yellow	Not Yellow	Total
Banana	400	100	350	150	450	50	500
Orange	0	300	150	150	300	0	300
Other	100	100	150	50	50	150	200
Total	500	500	650	350	800	200	1000

- So the objective of the classifier is to predict if a given fruit is a 'Banana' or 'Orange' or 'Other' when only the 3 features (long, sweet and yellow) are known.
- Let's say you are given a fruit that is: Long, Sweet and Yellow, can you predict what fruit it is?
- This is the same of predicting the Y when only the X variables in testing data are known. Let's solve it by hand using Naive Bayes.
- The idea is to compute the 3 probabilities, that is the probability of the fruit being a banana, orange or other. Whichever fruit type gets the highest probability wins.
- All the information to calculate these probabilities is present in the above tabulation.
- Step 1: Compute the 'Prior' probabilities for each of the class of fruits.
 - $P(Y=\text{Banana}) = 500 / 1000 = 0.50$
 - $P(Y=\text{Orange}) = 300 / 1000 = 0.30$
 - $P(Y=\text{Other}) = 200 / 1000 = 0.20$
- Step 2: Compute the probability of evidence that goes in the denominator.
 - $P(x_1=\text{Long}) = 500 / 1000 = 0.50$
 - $P(x_2=\text{Sweet}) = 650 / 1000 = 0.65$
 - $P(x_3=\text{Yellow}) = 800 / 1000 = 0.80$
- Step 3: Compute the probability of likelihood of evidences that goes in the numerator.
 - Here, I have done it for Banana alone.
 - Probability of Likelihood for Banana
 - $P(x_1=\text{Long} \mid Y=\text{Banana}) = 400 / 500 = 0.80$
 - $P(x_2=\text{Sweet} \mid Y=\text{Banana}) = 350 / 500 = 0.70$
 - $P(x_3=\text{Yellow} \mid Y=\text{Banana}) = 450 / 500 = 0.90$
- Step 4: Substitute all the 3 equations into the Naive Bayes formula, to get the probability that it is a banana.

Step 4: If a fruit is 'Long', 'Sweet' and 'Yellow', what fruit is it?

$$P(\text{Banana} | \text{Long, Sweet and Yellow}) = \frac{P(\text{Long} | \text{Banana}) * P(\text{Sweet} | \text{Banana}) * P(\text{Yellow} | \text{Banana}) * P(\text{Banana})}{P(\text{Long}) * P(\text{Sweet}) * P(\text{Yellow})}$$
$$= \frac{0.8 * 0.7 * 0.9 * 0.5}{P(\text{Evidence})} = 0.252 / P(\text{Evidence})$$
$$P(\text{Orange} | \text{Long, Sweet and Yellow}) = 0, \text{ because } P(\text{Long} | \text{Orange}) = 0$$
$$P(\text{Other Fruit} | \text{Long, Sweet and Yellow}) = 0.01875 / P(\text{Evidence})$$

Answer: Banana - Since it has highest probability amongst the 3 classes

Advantages

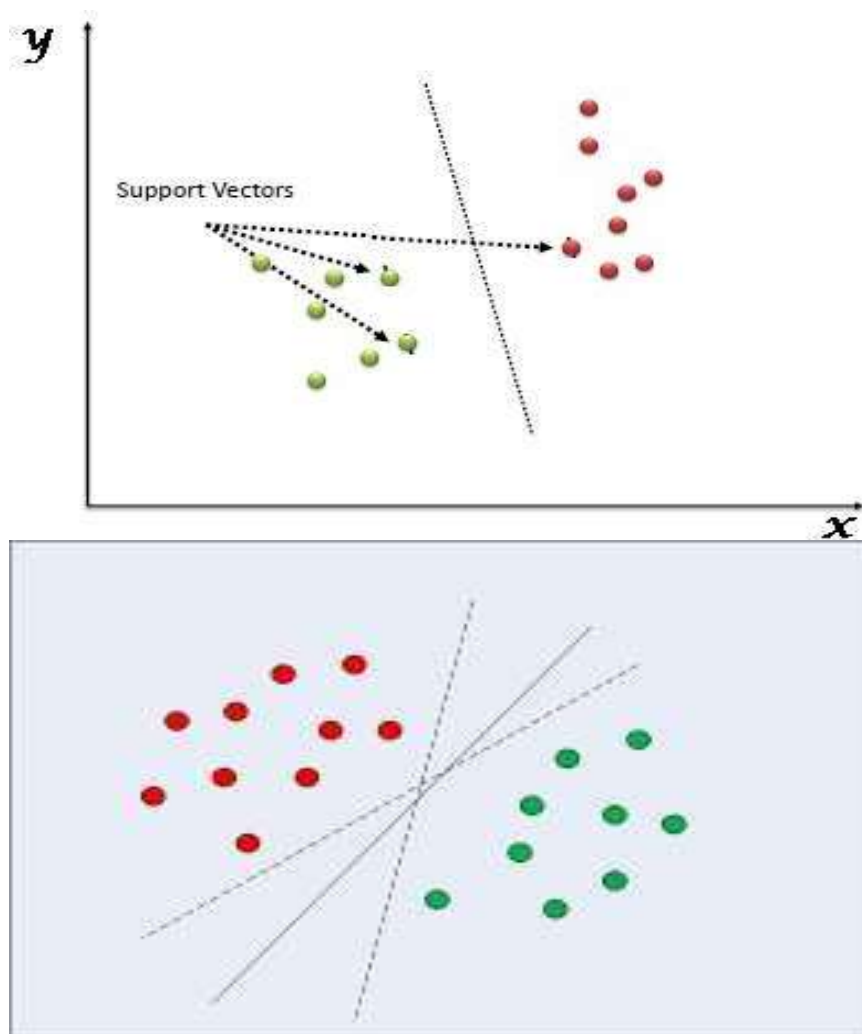
- When assumption of independent predictors holds true, a Naive Bayes classifier performs better as compared to other models
- Naive Bayes requires a small amount of training data to estimate the test data. So, the training period is less.
- Naive Bayes is also easy to implement.

Disadvantages

- Main imitation of Naive Bayes is the assumption of independent predictors. Naive Bayes implicitly assumes that all the attributes are mutually independent. In real life, it is almost impossible that we get a set of predictors which are completely independent.
- If categorical variable has a category in test data set, which was not observed in training data set, then model will assign a 0 (zero) probability and will be unable to make a prediction. This is often known as Zero Frequency. To solve this, we can use the smoothing technique. One of the simplest smoothing techniques is called Laplace estimation.

Support Vector Machine

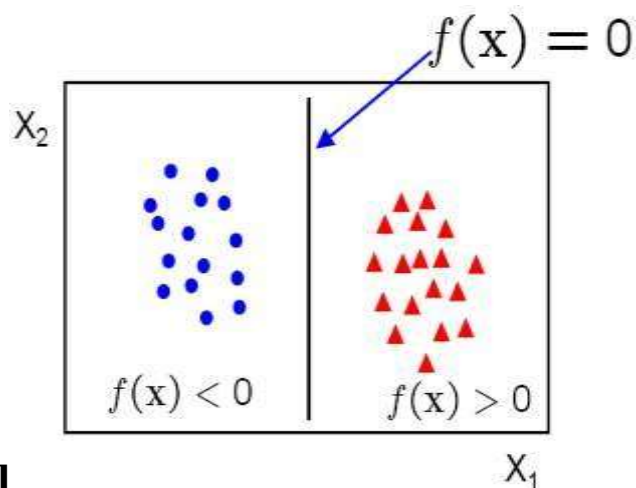
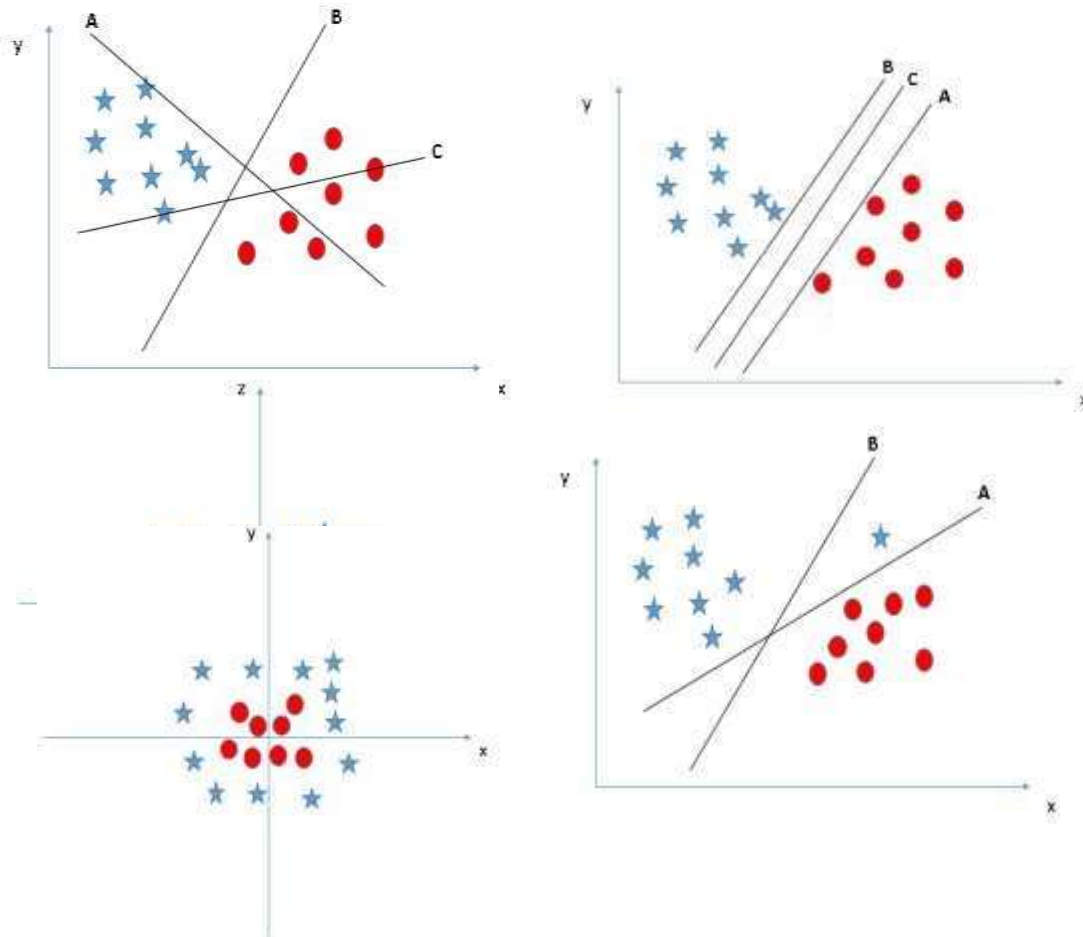
- “Support Vector Machine” (SVM) is a supervised machine learning algorithm which can be used for both classification or regression challenges.
- However, it is mostly used in classification problems.
- In this algorithm, we plot each data item as a point in n-dimensional space (where n is number of features you have) with the value of each feature being the value of a particular coordinate.
- Then, we perform classification by finding the hyper-plane that differentiate the two classes very well (look at the below snapshot).



How does it work?

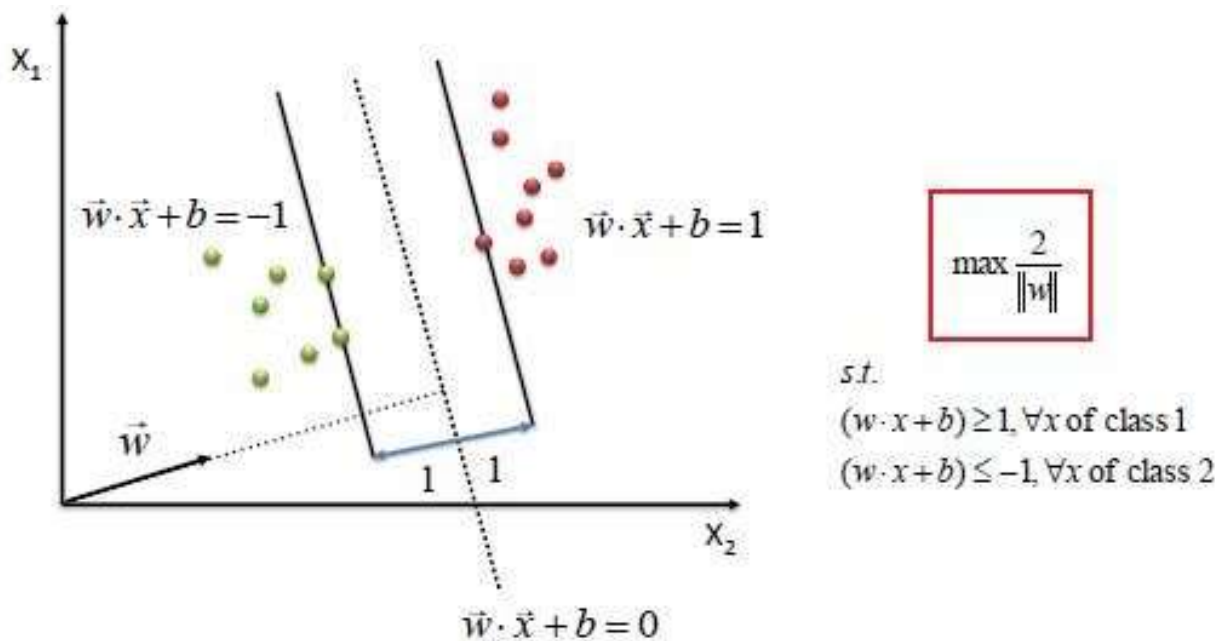
- Thumb rule to identify the right hyper-plane
- • Select the hyper-plane which segregates the two classes better
- • Maximizing the distances between nearest data point (either class) and hyper-plane.

This distance is called as Margin.



SVM Model

- $f(x) = W \cdot X + b$
- W is the normal to the line, X is input vector and b the bias
- W is known as the weight vector



Advantages of SVM

- The main strength of SVM is that they work well even when the number of SVM features is much larger than the number of instances.
- It can work on datasets with huge feature space, such is the case in spam filtering, where a large number of words are the potential signifiers of a message being spam.
- Even when the optimal decision boundary is a nonlinear curve, the SVM transforms the variables to create new dimensions such that the representation of the classifier is a linear function of those transformed dimensions of the data.
- SVMs are conceptually easy to understand. They create an easy-to-understand linear classifier. By working on only a subset of relevant data, they are computationally efficient. SVMs are now available with almost all data analytics toolsets.

Disadvantages of SVM

The SVM technique has two major constraints

- It works well only with real numbers, i.e., all the data points in all the dimensions must be defined by numeric values only,
- It works only with binary classification problems. One can make a series of cascaded

SVMs to get around this constraint.

- Training the SVMs is an inefficient and time consuming process, when the data is large.
- It does not work well when there is much noise in the data, and thus has to compute soft margins.
- The SVMs will also not provide a probability estimate of classification, i.e., the confidence level for classifying an instance.

Web Mining

Web mining is the art and science of discovering patterns and insights from the World-wide web so as to improve it. The world-wide web is at the heart of the digital revolution. More data is posted on the web every day than was there on the whole web just 20 years ago. Billions of users are using it every day for a variety of purposes. The web is used for electronic commerce, business communication, and many other applications. Web mining analyzes data from the web and helps find insights that could optimize the web content and improve the user experience. Data for web mining is collected via Web crawlers, web logs, and other means.

Here are some characteristics of optimized websites:

1. *Appearance*: Aesthetic design. Well-formatted content, easy to scan and navigate. Good color contrasts.
2. *Content*: Well planned information architecture with useful content. Fresh content. Search-engine optimized. Links to other goodsites.
3. *Functionality*: Accessible to all authorized users. Fast loading times. Usable forms. Mobile enabled.

This type of content and its structure is of interest to ensure the web is easy to use. The analysis of web usage provides feedback on the web content, and also the consumer's browsing habits. This data can be of immense use for commercial advertising, and even for social engineering.

The web could be analyzed for its structure as well as content. The usage pattern of web pages could also be analyzed. Depending upon objectives, web mining can be divided into three different types: Web usage mining, Web content mining and Web structure mining (Figure 2.1).

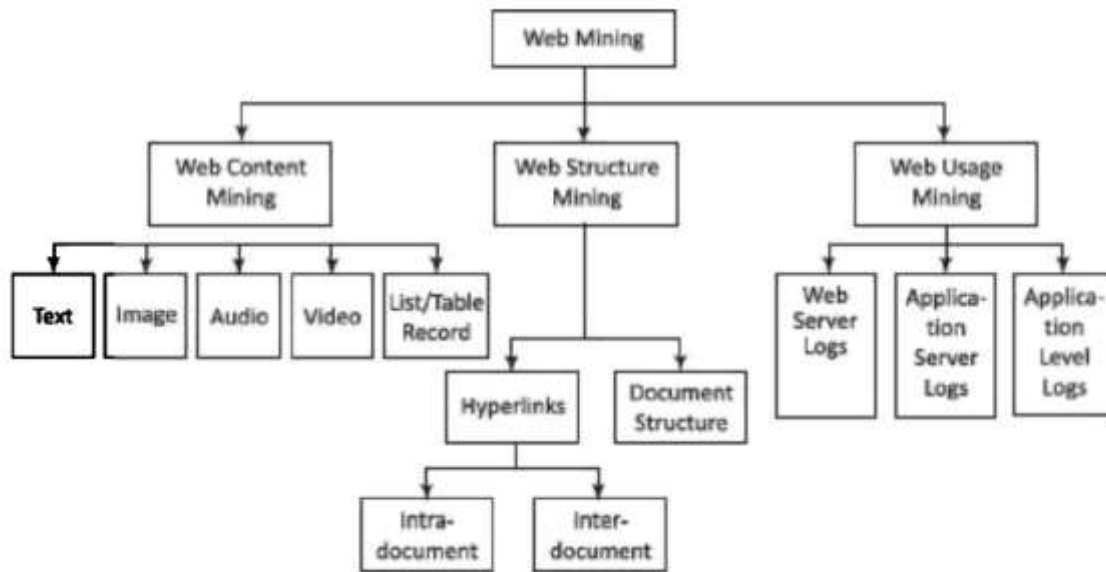


Figure: 2.1 Web Mining structure/Taxonomy

Web content mining

A website is designed in the form of pages with a distinct URL (universal resource locator). A large website may contain thousands of pages. These pages and their content is managed using specialized software systems called Content Management Systems. Every page can have text, graphics, audio, video, forms, applications, and more kinds of content including user generated content.

The websites keep a record of all requests received for its page/URLs, including the requester information using 'cookies'. The log of these requests could be analyzed to gauge the popularity of those pages among different segments of the population. The text and application content on the pages could be analyzed for its usage by visit counts. The pages on a website themselves could be analyzed for quality of content that attracts most users. Thus the unwanted or unpopular pages could be weeded out, or they can be transformed with different content and style. Similarly, more resources could be assigned to keep the more popular pages more fresh and inviting.

Common Web Content Mining Techniques:

Pre-processing of contents The pre-processing steps are quite similar to the pre-processing for text mining. The content preparation involves:

1. Extraction of text from HTML
2. Data cleaning by filling up the missing values and smoothing the noisy data
3. Tokenizing: Generates the tokens of words from the cleaned up text
4. *Stemming*: Reduce the words to their roots. The different grammatical forms or declinations of verbs identify and index (count) as the same word. For example, stemming will ensure that both “closed” and “closing” are derived from the same word “close”. Stemming algorithm, *Porter*, can be used here. The java code for *Porter* stemming algorithm can be obtained from <https://tartarus.org/martin/PorterStemmer/java.txt>,
5. *Removing the stop words*: The common words unlikely to help in the mining process such as articles (a, an, the), or prepositions (such as, to, in, for) are removed.
6. *Calculate collection wide-word frequencies*: The distinct-word stem obtained after stemming process and removing the stop words results into a list of significant words (or terms). Calculating the occurrence of a significant term (t) in a collection is called collection frequency (CF_t). CF counts the multiple occurrences.)

Now, find the number of documents in the collection that contains the specific term (t). This numeric measure is the document frequency (DF_t).
7. *Calculate per Document Term Frequencies* (TF). TF is a numeric measure that is used to score the importance of a word in a document based on how often it appeared in that document (Refer Example 9.1).
8. *Bag of words*: Web document is represented by the words it contains (and their occurrences).

The following example explains the concept of CF and DF using the data of toy sales collection.

Web structure mining

The Web works through a system of hyperlinks using the hypertext protocol (http). Any page can create a hyperlink to any other page, it can be linked to by another page. The intertwined or self-referral nature of web lends itself to some unique network analytical algorithms. The structure of Web pages could also be analyzed to examine the pattern of hyperlinks among pages. There are two basic strategic models for successful websites: Hubs and Authorities.

1. *Hubs*: These are pages with a large number of interesting links. They serve as a hub, or a gathering point, where people visit to access a variety of information. Media sites like Yahoo.com, or government sites would serve that purpose. More focused sites like Traveladvisor.com and yelp.com could aspire to becoming hubs for new emerging areas.
2. *Authorities*: Ultimately, people would gravitate towards pages that provide the most complete and authoritative information on a particular subject. This could be factual

information, news, advice, user reviews etc. These websites would have the most number of inbound links from other websites. Thus Mayoclinic.com would serve as an authoritative page for expert medical opinion. NYtimes.com would serve as an authoritative page for daily news.

Web usage mining

As a user clicks anywhere on a webpage or application, the action is recorded by many entities in many locations. The browser at the client machine will record the click, and the web server providing the content would also make a record of the pages served and the user activity on those pages. The entities between the client and the server, such as the [router](#), [proxy server](#), or [ad server](#), too would record that click.

The goal of web usage mining is to extract useful information and patterns from data generated through Web page visits and transactions. The activity data comes from data stored in server access logs, referrer logs, agent logs, and client-side cookies. The user characteristics and usage profiles are also gathered directly, or indirectly, through syndicated data. Further, metadata, such as page attributes, content attributes, and usage data are also gathered.

The web content could be analyzed at multiple levels (Figure 2.2).

1. The *server side analysis* would show the relative popularity of the web pages accessed. Those websites could be hubs and authorities.
2. The *client side analysis* could focus on the usage pattern or the actual content consumed and created by users.
 1. Usage pattern could be analyzed using ‘clickstream’ analysis, i.e. analyzing web activity for patterns of sequence of clicks, and the location and duration of visits on websites. Clickstream analysis can be useful for web activity analysis, software testing, market research, and analyzing employee productivity.
 2. Textual information accessed on the pages retrieved by users could be analyzed using text mining techniques. The text would be gathered and structured using the bag-of-words technique to build a Term-document matrix. This matrix could then be mined using cluster analysis and association rules for patterns such as popular topics, user segmentation, and sentiment analysis.

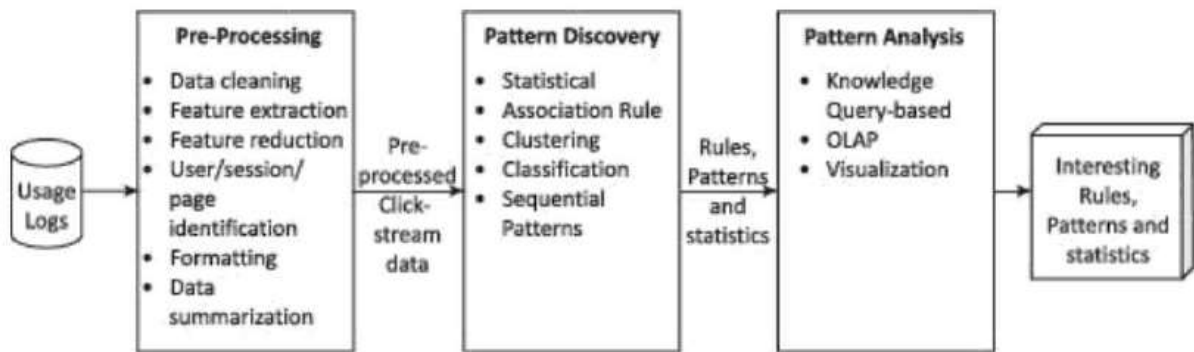


Figure: 2.2 Web Usage Mining Architecture

Web usage mining has many business applications. It can help predict user behavior based on previously learned rules and users' profiles, and can help determine lifetime value of clients. It can also help design cross-marketing strategies across products, by observing association rules among the pages on the website. Web usage can help evaluate promotional campaigns and see if the users were attracted to the website and used the pages relevant to the campaign. Web usage mining could be used to present dynamic information to users based on their interests and profiles. This includes targeted online ads and coupons at user groups based on user access patterns.

Web Mining Algorithms

Hyperlink-Induced Topic Search (HITS) is a link analysis algorithm that rates web pages as being hubs or authorities. Many other HITS-based algorithms have also been published. The most famous and powerful of these algorithms is the PageRank algorithm. Invented by Google co-founder Larry Page, this algorithm is used by Google to organize the results of its search function. This algorithm helps determine the relative importance of any particular web page by counting the number and quality of links to a page. The websites with more number of links, and/or more links from higher-quality websites, will be ranked higher. It works in a similar way as determining the status of a person in a society of people. Those with relations to more people and/or relations to people of higher status will be accorded a higher status.

PageRank is the algorithm that helps determine the order of pages listed upon a Google Search query. The original PageRank algorithm formulation has been updated in many ways and the latest algorithm is kept a secret so other websites cannot take advantage of the algorithm and manipulate their website according to it. However, there are many standard elements that remain unchanged. These elements lead to the principles for a good website. This process is also called Search Engine Optimization (SEO).

HUB and AUTHORITIES

Hub:

An index page that out-links to many content pages.

It acts like a directory pointing to useful resources on a topic.

Authority:

A content page that has been linked to by many hubs.

It is recognized for having useful, reliable, and significant information on a topic.

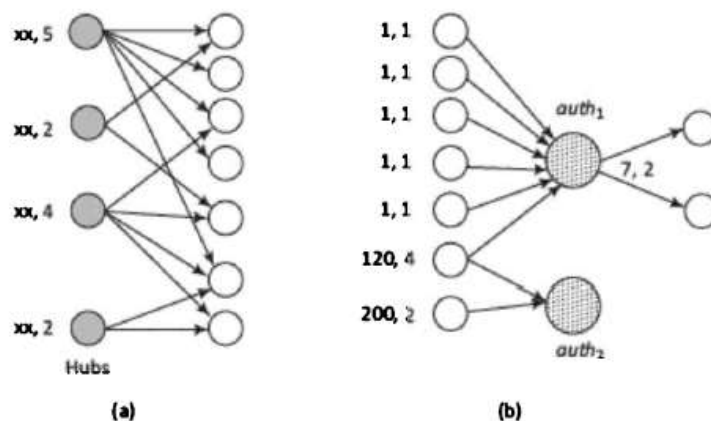


Figure 9.10 (a) Hubs (shaded circles) and (b) Authorities (dotted circles)